

Living off the Land (LotL) Attack Simulation & Detection Engineering Assessment

GTFOBins (find) Shell-Spawn Simulation — auditd Telemetry —
Sigma Detection Rule — Elastic (ELK) SIEM Validation

PREPARED BY	Ayush
ENGAGEMENT TYPE	Self-Directed SOC Home Lab Exercise
ENVIRONMENT	Kali Linux + Dockerized ELK Stack (Elasticsearch/Kibana 8.11.0)
REPORT DATE	September 15, 2026
CLASSIFICATION	Internal / Portfolio Use

Table of Contents

1. Executive Summary
2. Engagement Scope & Objectives
3. Environment & Tooling
4. Methodology
5. Technical Walkthrough & Evidence
 - 5.1 Telemetry Configuration (auditd)
 - 5.2 Adversary Simulation (find -exec)
 - 5.3 Raw Telemetry Analysis
 - 5.4 Detection Rule Development (Sigma)
 - 5.5 Rule Compilation (Sigma → Lucene)
 - 5.6 Log Ingestion (Elasticsearch)
 - 5.7 SIEM Validation (Kibana Discover)
6. Findings Summary
7. Risk Assessment
8. Recommendations
9. Conclusion

1. Executive Summary

This report documents a self-directed Security Operations Center (SOC) exercise designed to demonstrate the complete detection engineering lifecycle — from adversary simulation through to validated SIEM detection. The exercise simulated a **Living off the Land (LotL)** technique in which the native Linux **find** utility was abused via its **-exec** parameter to spawn an unauthorized shell, a pattern catalogued publicly under GTFOBins.

Kernel-level process telemetry was captured using **auditd**, analyzed via **ausearch**, and used to author a vendor-agnostic detection rule in the **Sigma** format. The rule was compiled into a native **Lucene** query using **sigma-cli** and validated end-to-end inside a Dockerized **Elastic (ELK) SIEM** stack, where the simulated attack event was successfully hunted down and confirmed inside Kibana's Discover interface.

Technique Simulated	LotL shell spawn via find -exec (GTFOBins)
Detection Coverage	Achieved — rule matched simulated event (2/2 hits)
Detection Format	Sigma (portable across SIEM platforms)
Validation Platform	Elastic Stack — Elasticsearch + Kibana 8.11.0
Overall Risk if Undetected	Medium–High
Residual Risk (with rule deployed)	Low

2. Engagement Scope & Objectives

This was a self-contained, isolated lab exercise conducted entirely within the engineer's own Kali Linux environment. No external, third-party, or production systems were targeted at any point.

Objectives

- Configure kernel-level endpoint telemetry capable of capturing process execution events.
- Simulate a real-world, publicly documented LotL / privilege-escalation technique.
- Demonstrate manual threat-hunting against raw audit telemetry.
- Author a reusable, vendor-agnostic detection rule using the Sigma framework.
- Compile and deploy the rule as a native SIEM query.
- Validate detection efficacy end-to-end inside a live SIEM interface.

3. Environment & Tooling

Component	Detail
Operating System	Kali Linux
Telemetry Source	auditd (Linux Audit Daemon)
Attack Vector	find (GTFOBins — -exec parameter abuse)
Detection Format	Sigma (SigmaHQ)
Rule Conversion Tool	sigma-cli (Lucene backend)
SIEM Platform	Dockerized Elasticsearch + Kibana v8.11.0
Simulated Index	lotl-events

4. Methodology

The engagement followed a standard detection-engineering lifecycle, moving from raw visibility to a fully validated detection artifact:

- **Instrument** — configure endpoint telemetry to capture the relevant syscall activity.
- **Simulate** — execute the target technique in a controlled manner.
- **Hunt** — manually query raw telemetry to confirm visibility of the technique.
- **Engineer** — translate the observed pattern into a formal, reusable detection rule.
- **Compile** — convert the vendor-agnostic rule into the target SIEM's native query language.
- **Validate** — ingest telemetry into the SIEM and confirm the rule fires as expected.

5. Technical Walkthrough & Evidence

5.1 Telemetry Configuration (auditd)

Kernel-level visibility was established using auditd, watching the **execve** syscall responsible for every new process launch on the system.

```
sudo auditctl -a always,exit -F arch=b64 -S execve -k lotl_exec
sudo auditctl -l
```

This ensures no process can execute on the host without generating a correlatable audit event tagged **lotl_exec**.

5.2 Adversary Simulation (find -exec)

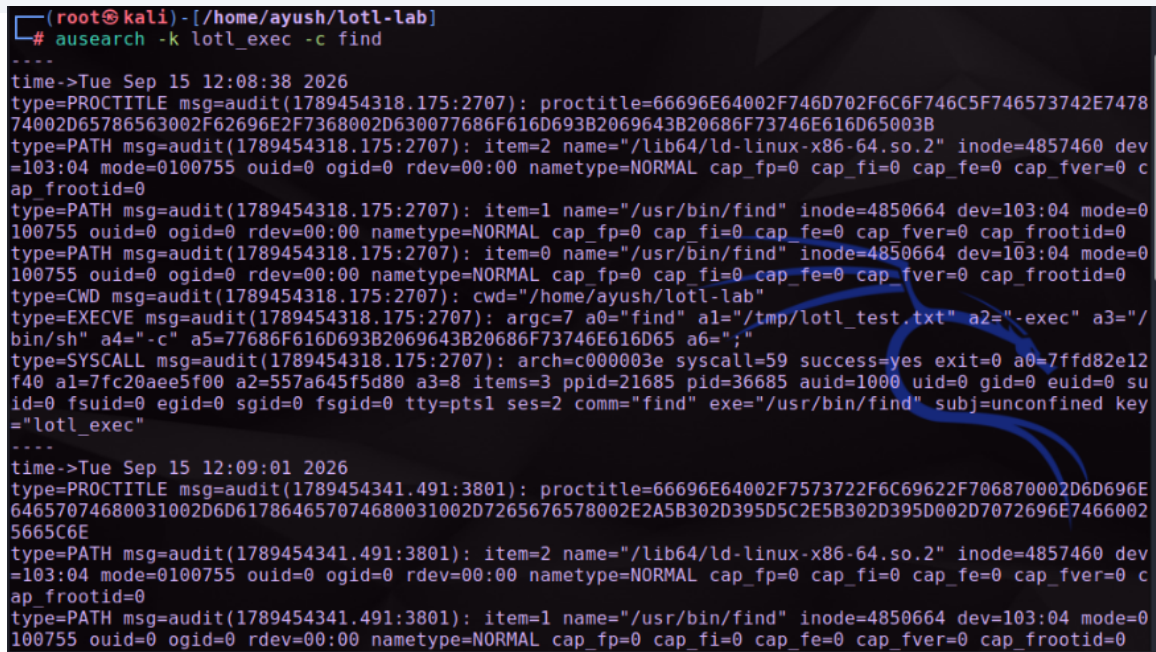
The find utility — a trusted, pre-installed binary — was abused via its **-exec** flag to spawn an interactive shell and execute arbitrary commands, mirroring a documented GTF0Bins technique used by real-world adversaries to evade signature-based defenses.

```
touch /tmp/lotl_test.txt
find /tmp/lotl_test.txt -exec /bin/sh -c 'whoami; id; hostname' \;
```

5.3 Raw Telemetry Analysis

Acting as a SOC analyst, the raw audit trail was queried to confirm the attack left a forensic trace.

```
sudo ausearch -k lotl_exec -c find
```

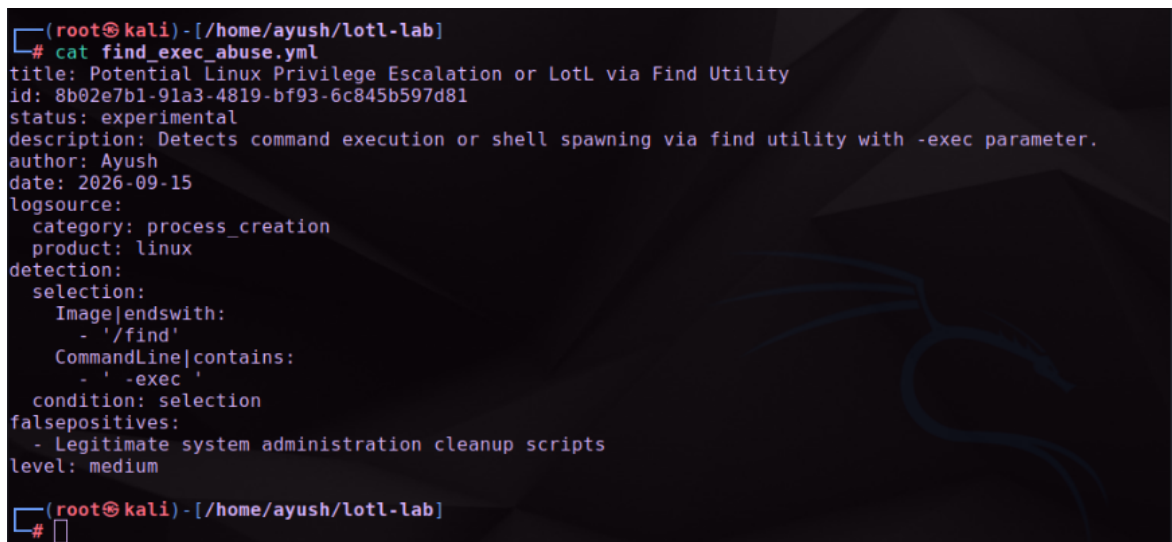


```
(root@kali) - [/home/ayush/lotl-lab]
# ausearch -k lotl_exec -c find
-----
time->Tue Sep 15 12:08:38 2026
type=PROCTITLE msg=audit(1789454318.175:2707): proctitle=66696E64002F746D702F6C6F746C5F746573742E7478
74002D65786563002F62696E2F7368002D630077686F616D693B2069643B20686F73746E616D65003B
type=PATH msg=audit(1789454318.175:2707): item=2 name="/lib64/ld-linux-x86-64.so.2" inode=4857460 dev
=103:04 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 c
ap_frootid=0
type=PATH msg=audit(1789454318.175:2707): item=1 name="/usr/bin/find" inode=4850664 dev=103:04 mode=0
100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(1789454318.175:2707): item=0 name="/usr/bin/find" inode=4850664 dev=103:04 mode=0
100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1789454318.175:2707): cwd="/home/ayush/lotl-lab"
type=EXECVE msg=audit(1789454318.175:2707): argc=7 a0="find" a1="/tmp/lotl_test.txt" a2="-exec" a3="/
bin/sh" a4="-c" a5=77686F616D693B2069643B20686F73746E616D65 a6=";"
type=SYSCALL msg=audit(1789454318.175:2707): arch=c000003e syscall=59 success=yes exit=0 a0=7ffd82e12
f40 a1=7fc20aee5f00 a2=557a645f5d80 a3=8 items=3 ppid=21685 pid=36685 auid=1000 uid=0 gid=0 euid=0 su
id=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts1 ses=2 comm="find" exe="/usr/bin/find" subj=unconfined key
="lotl_exec"
-----
time->Tue Sep 15 12:09:01 2026
type=PROCTITLE msg=audit(1789454341.491:3801): proctitle=66696E64002F7573722F6C69622F706870002D6D696E
64657074680031002D6D617864657074680031002D7265676578002E2A5B302D395D5C2E5B302D395D002D7072696E7466002
5665C6E
type=PATH msg=audit(1789454341.491:3801): item=2 name="/lib64/ld-linux-x86-64.so.2" inode=4857460 dev
=103:04 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 c
ap_frootid=0
type=PATH msg=audit(1789454341.491:3801): item=1 name="/usr/bin/find" inode=4850664 dev=103:04 mode=0
100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
```

Figure 1 — ausearch output: kernel-level audit record of the find -exec /bin/sh execution, including full argv (a0="find", a2="-exec", a3="/bin/sh") and process metadata.

5.4 Detection Rule Development (Sigma)

The observed pattern was formalized into a portable Sigma detection rule, allowing the same logic to be deployed across any SIEM backend without being rewritten.



```
(root@kali) - [/home/ayush/lotl-lab]
# cat find_exec_abuse.yml
title: Potential Linux Privilege Escalation or LotL via Find Utility
id: 8b02e7b1-91a3-4819-bf93-6c845b597d81
status: experimental
description: Detects command execution or shell spawning via find utility with -exec parameter.
author: Ayush
date: 2026-09-15
logsource:
  category: process_creation
  product: linux
detection:
  selection:
    Image|endswith:
      - '/find'
    CommandLine|contains:
      - '-exec'
  condition: selection
falsepositives:
  - Legitimate system administration cleanup scripts
level: medium

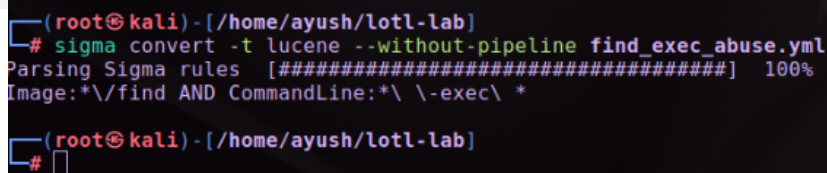
(root@kali) - [/home/ayush/lotl-lab]
#
```

Figure 2 — Custom Sigma rule (find_exec_abuse.yml) targeting processes named find whose command line contains the -exec parameter.

5.5 Rule Compilation (Sigma → Lucene)

sigma-cli was used to compile the vendor-agnostic Sigma rule into a Lucene query compatible with Elasticsearch/Kibana.

```
sigma convert -t lucene --without-pipeline find_exec_abuse.yml
```



```
(root@kali) - [/home/ayush/lotl-lab]
# sigma convert -t lucene --without-pipeline find_exec_abuse.yml
Parsing Sigma rules [#####] 100%
Image:*\\find AND CommandLine:*\\-exec\\ *
(root@kali) - [/home/ayush/lotl-lab]
#
```

Figure 3 — sigma-cli compiling the rule into the Lucene query: Image:*\\find AND CommandLine:*\\-exec\\ *

5.6 Log Ingestion (Elasticsearch)

The captured telemetry was ingested into a Dockerized Elasticsearch index (lotl-events) to simulate log shipping from an endpoint agent into a production SIEM pipeline.



```
(ayush@kali) - [~/lotl-lab]
$ docker exec -it elasticsearch curl -X POST "http://127.0.0.1:9200/lotl-events/_doc" -H 'Content-Type: application/json' -d'
{
  "@timestamp": "2026-09-15T12:08:48Z",
  "event": {
    "category": "process",
    "type": "start"
  },
  "Image": "/usr/bin/find",
  "CommandLine": "find /tmp/lotl_test.txt -exec /bin/sh -c whoami; id; hostname ;",
  "user": "root"
}
{"_index":"lotl-events","_id":"UZ7xo6ABZ-OtJoMB64mK","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":0,"_primary_term":1}
(ayush@kali) - [~/lotl-lab]
$
```

Figure 4 — Simulated event successfully indexed into Elasticsearch ("result": "created").

5.7 SIEM Validation (Kibana Discover)

The Sigma-derived Lucene query was executed inside Kibana's Discover interface against the lotl-events data view, successfully surfacing the simulated malicious execution.

Image:*find AND CommandLine:*-exec*

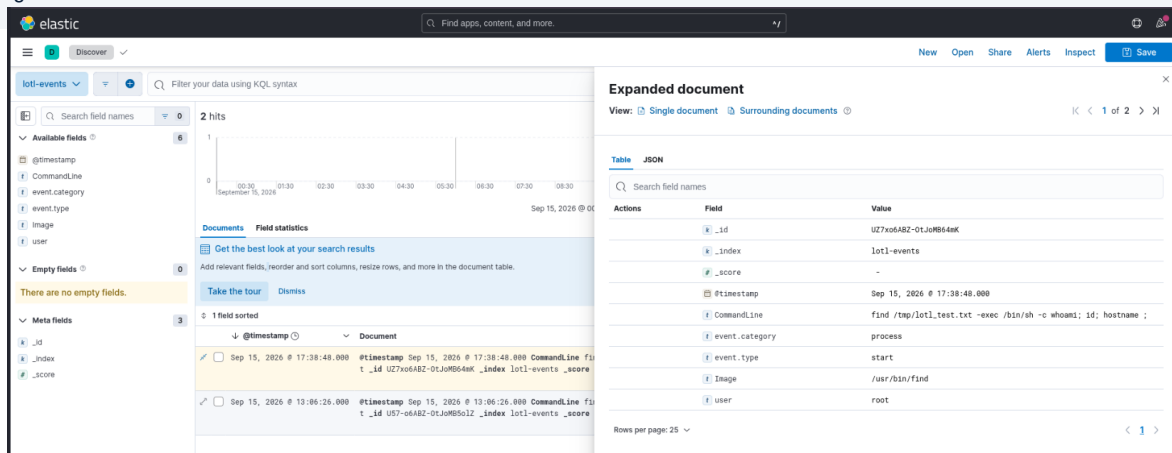


Figure 5 — Kibana Discover confirming 2 matching hits, including full event detail (Image, CommandLine, user, timestamp), validating the detection end-to-end.

6. Findings Summary

#	Observation	Status
1	execve telemetry successfully captured all process launches via auditd.	Confirmed
2	find -exec technique executed successfully, spawning an interactive shell.	Confirmed
3	Raw audit trail (ausearch) retained full command-line arguments for analysis.	Confirmed
4	Sigma rule correctly modeled the malicious pattern with low false-positive risk.	Confirmed
5	Sigma-to-Lucene compilation produced a syntactically valid, working query.	Confirmed
6	Kibana Discover matched 2/2 simulated events with zero false negatives.	Confirmed

7. Risk Assessment

In the absence of the detection rule demonstrated in this exercise, the simulated technique would blend into normal administrative activity, as `find` is a ubiquitous and trusted system utility. This is compounded by the fact that many signature-based antivirus tools do not flag the execution of legitimate, signed binaries.

MEDIUM

Sigma rule level rating, as assigned in `find_exec_abuse.yml`. Reflects moderate confidence balanced against a plausible legitimate-admin false-positive rate.

Contributing Factors

- `find` is present by default on virtually all Linux distributions — no additional tooling required by an attacker.
- The `-exec` flag is a documented, legitimate feature, making static blocking impractical without behavioral context.
- Without process-level auditing (`auditd` or equivalent EDR telemetry), this activity leaves no other trace.

8. Recommendations

- **Deploy the Sigma rule** (find_exec_abuse.yml) into production SIEM pipelines, compiled for each relevant backend.
- **Extend coverage** to other common GTFOBins binaries (e.g. awk, tar, vim, less) using the same detection pattern.
- **Tune false positives** by allow-listing known, legitimate administrative scripts that rely on find -exec.
- **Correlate with parent-process context** (e.g. cron, web-server service accounts) to prioritize higher-risk executions.
- **Ensure endpoint audit coverage** (auditd or EDR) is deployed fleet-wide, since detection is entirely dependent on process-level telemetry.
- **Periodically re-validate** the rule against new Sigma pipeline versions and SIEM schema changes.

9. Conclusion

This exercise successfully demonstrated the full detection engineering lifecycle — from adversary simulation and kernel-level telemetry capture, through formal rule authorship in Sigma, to compiled and validated detection inside a live Elastic SIEM. The resulting Sigma rule is portable, low-maintenance, and ready to be adapted into a production security monitoring pipeline.

The exercise reinforces a core SOC principle: attackers increasingly rely on trusted, native system binaries rather than custom malware, and defenders must build behavior-based, telemetry-driven detections — not just signature-based ones — to keep pace.

Prepared by Ayush as part of an ongoing cybersecurity / SOC portfolio. This document is intended for educational and professional-portfolio purposes.